

4. Implementation of the following Memory Allocation Methods for fixed partition a. First Fit b. Worst Fit c. Best Fit

```
import random
import matplotlib.pyplot as plt
import pandas as pd

method = input("Enter allocation method (first / best / worst): ").lower()
n_blocks = int(input("Enter number of blocks: "))
n_files = int(input("Enter number of files: "))

block_status = [random.randint(0, 1) for _ in range(n_blocks)]
block_sizes = [random.randint(10, 60) if b == 0 else 0 for b in block_status]
file_sizes = [random.randint(5, 30) for _ in range(n_files)]

allocation = [-1] * n_files
fragment = [0] * n_files
used = [False] * n_blocks

free_blocks = [(block_sizes[i], i) for i in range(n_blocks) if block_status[i] == 0]

if method == "best":
    free_blocks.sort()
elif method == "worst":
    free_blocks.sort(reverse=True)

for i in range(n_files):
    for size, idx in free_blocks:
        if size >= file_sizes[i] and not used[idx]:
            allocation[i] = idx
            fragment[i] = size - file_sizes[i]
            used[idx] = True
            break

print("\nFile_no File_size Block_no Block_size Fragment")
print("-"*50)
for i in range(n_files):
    if allocation[i] != -1:
        print(f'{i+1:<8} {file_sizes[i]:<11} {'B'+str(allocation[i]+1):<10} {block_sizes[allocation[i]]:<12} {fragment[i]}")
```

```

else:
    print(f'{i+1:<8} {file_sizes[i]:<11} {'NA':<10} {'NA':<12} {'NA'}")

block_to_file = {}
for i, b in enumerate(allocation):
    if b != -1:
        block_to_file[b] = f'P {i+1}'

fig, ax = plt.subplots(figsize=(4, 9))
ax.set_xlim(0, 1)
ax.set_ylim(0, n_blocks)
ax.axis("off")

for i in range(n_blocks):
    y = n_blocks - i - 1

    ax.text(-0.05, y + 0.5, f'B {i+1}', ha="right", va="center", weight="bold")

    if block_status[i] == 1:
        ax.add_patch(plt.Rectangle((0, y), 1, 1, color="yellow", ec="black"))
        ax.text(0.5, y + 0.5, "USED", ha="center", va="center", weight="bold")

    elif used[i]:
        ax.add_patch(plt.Rectangle((0, y), 1, 1, color="green", ec="black"))
        label = block_to_file.get(i, "")
        ax.text(0.5, y + 0.5, f'{label}\n{block_sizes[i]} KB',
                ha="center", va="center", fontsize=10, weight="bold")

    else:
        ax.add_patch(plt.Rectangle((0, y), 1, 1, color="white", ec="black"))
        ax.text(0.5, y + 0.5, f'{block_sizes[i]} KB', ha="center", va="center")

ax.set_title(
    f'Memory Allocation using {method.upper()} Fit (OS View)',
    fontsize=13, fontweight="bold", pad=15
)

plt.show()

```

Output:

Enter allocation method (first / best / worst): first
Enter number of blocks: 20
Enter number of files: 5

File_no	File_size	Block_no	Block_size	Fragment
1	6	B1	22	16
2	10	B2	42	32
3	5	B3	53	48
4	7	B8	34	27
5	17	B9	26	9

Memory Allocation using FIRST Fit (OS View)

B1	P1 22 KB
B2	P2 42 KB
B3	P3 53 KB
B4	USED
B5	USED
B6	USED
B7	USED
B8	P4 34 KB
B9	P5 26 KB
B10	USED
B11	USED
B12	34 KB
B13	25 KB
B14	USED
B15	28 KB
B16	USED
B17	USED
B18	USED
B19	52 KB
B20	11 KB

Enter allocation method (first / best / worst): best
Enter number of blocks: 20
Enter number of files: 5

File_no	File_size	Block_no	Block_size	Fragment
1	21	B16	25	4
2	15	B10	34	19
3	24	B13	41	17
4	12	B6	47	35
5	30	B19	48	18

Memory Allocation using BEST Fit (OS View)

B1	USED
B2	USED
B3	USED
B4	USED
B5	USED
B6	P4 47 KB
B7	52 KB
B8	USED
B9	USED
B10	P2 34 KB
B11	USED
B12	USED
B13	P3 41 KB
B14	USED
B15	58 KB
B16	P1 25 KB
B17	USED
B18	49 KB
B19	P5 48 KB
B20	52 KB

Enter allocation method (first / best / worst): worst
Enter number of blocks: 20
Enter number of files: 5

File_no	File_size	Block_no	Block_size	Fragment

1	25	B8	56	31
2	30	B13	53	23
3	6	B6	41	35
4	10	B2	40	30
5	20	B11	31	11

Memory Allocation using WORST Fit (OS View)

B1	USED
B2	P4 40 KB
B3	USED
B4	USED
B5	24 KB
B6	P3 41 KB
B7	USED
B8	P1 56 KB
B9	USED
B10	USED
B11	P5 31 KB
B12	USED
B13	P2 53 KB
B14	17 KB
B15	USED
B16	USED
B17	15 KB
B18	USED
B19	USED
B20	USED

**6. Implementation of the following Page Replacement Algorithms a. FIFO
b. LRU c. optimal**

```
import random
import matplotlib.pyplot as plt

def get_victim_index(algo, frames, pages, current_idx, recent_usage, fifo_q):
    if algo == "FIFO":
        return fifo_q, (fifo_q + 1) % len(frames)

    if algo == "LRU":
        victim = min(frames, key=lambda x: recent_usage[x])
        return frames.index(victim), fifo_q

    if algo == "OPTIMAL":
        future = pages[current_idx+1:]
        victim = max(frames, key=lambda x: future.index(x) if x in future else float('inf'))
        return frames.index(victim), fifo_q

def simulate(pages, cap, algo):
    frames, history, status, repl_str = [], [], [], []
    faults, fifo_q = 0, 0
    recent = {}

    for i, p in enumerate(pages):
        recent[p] = i

        if p in frames:
            status.append("HIT"); repl_str.append("-")
        else:
            faults += 1
            status.append("FAULT")

            if len(frames) < cap:
                frames.append(p)
                repl_str.append(f"->{p}")
            else:
                idx, fifo_q = get_victim_index(algo, frames, pages, i, recent, fifo_q)
                repl_str.append(f"{frames[idx]}->{p}")
                frames[idx] = p
```

```

        history.append(frames[:])

    return faults, history, status, repl_str

try:
    n = int(input("Ref string size: "))
    cap = int(input("Frame slots: "))
    choice = int(input("1.FIFO 2.LRU 3.OPTIMAL\nChoice: "))
    algos = {1: "FIFO", 2: "LRU", 3: "OPTIMAL"}
    algo = algos[choice]
except (ValueError, KeyError):
    exit("Invalid Input")

pages = [random.randint(1, 9) for _ in range(n)]
faults, history, status, repl = simulate(pages, cap, algo)

print(f"\nAlgorithm: {algo}\nFrames: {cap}\nTotal Pages: {n}\nHits: {n-faults}\nFaults: {faults}\nRef String: {pages}")

table_data = [
    [h[i] if i < len(h) else "" for h in history]
    for i in range(cap)
] + [status, repl]

row_labels = [f"Frame {i+1}" for i in range(cap)] + ["Status", "Change"]

plt.figure(figsize=(max(10, n*0.8), 5))
plt.axis('off')
plt.title(f'{algo} Page Replacement Simulation', fontsize=14, weight='bold')
tbl = plt.table(cellText=table_data, rowLabels=row_labels, colLabels=list(map(str, pages)),
loc='center', cellLoc='center')
tbl.scale(1, 1.5)
plt.tight_layout()
plt.show()

```

VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ Date: _____

Output:

Ref string size: 20
Frame slots: 4
1.FIFO 2.LRU 3.OPTIMAL
Choice: 1

Algorithm: FIFO
Frames: 4
Total Pages: 20
Hits: 11
Faults: 9

Ref String: [5, 4, 9, 6, 3, 6, 3, 9, 7, 5, 4, 6, 5, 5, 4, 7, 5, 5, 5, 5]

FIFO Page Replacement Simulation

	5	4	9	6	3	6	3	9	7	5	4	6	5	5	4	7	5	5	5	5
Frame 1	5	5	5	5	3	3	3	3	3	3	3	6	6	6	6	6	6	6	6	6
Frame 2		4	4	4	4	4	4	4	7	7	7	7	7	7	7	7	7	7	7	7
Frame 3			9	9	9	9	9	9	5	5	5	5	5	5	5	5	5	5	5	5
Frame 4				6	6	6	6	6	6	4	4	4	4	4	4	4	4	4	4	4
Status	FAULT	FAULT	FAULT	FAULT	FAULT	HIT	HIT	HIT	FAULT	FAULT	FAULT	FAULT	HIT	HIT	HIT	HIT	HIT	HIT	HIT	HIT
Change	->5	->4	->9	->6	5->3	-	-	-	4->7	9->5	6->4	3->6	-	-	-	-	-	-	-	-

Ref string size: 20
Frame slots: 4
1.FIFO 2.LRU 3.OPTIMAL
Choice: 2

Algorithm: LRU
Frames: 4
Total Pages: 20
Hits: 4
Faults: 16

Ref String: [6, 4, 5, 7, 1, 6, 6, 8, 8, 4, 2, 4, 5, 6, 1, 8, 2, 5, 1, 6]

LRU Page Replacement Simulation

	6	4	5	7	1	6	6	8	8	4	2	4	5	6	1	8	2	5	1	6
Frame 1	6	6	6	6	1	1	1	1	1	1	2	2	2	2	1	1	1	1	1	1
Frame 2		4	4	4	4	6	6	6	6	6	6	6	5	5	5	5	2	2	2	2
Frame 3			5	5	5	5	5	8	8	8	8	8	8	6	6	6	6	5	5	5
Frame 4				7	7	7	7	7	7	4	4	4	4	4	4	8	8	8	8	6
Status	FAULT	FAULT	FAULT	FAULT	FAULT	FAULT	HIT	FAULT	HIT	FAULT	FAULT	HIT	FAULT	FAULT	FAULT	FAULT	FAULT	FAULT	HIT	FAULT
Change	->6	->4	->5	->7	6->1	4->6	-	5->8	-	7->4	1->2	-	6->5	8->6	2->1	4->8	5->2	6->5	-	8->6

Ref string size: 20
Frame slots: 4
1.FIFO 2.LRU 3.OPTIMAL
Choice: 3

Algorithm: OPTIMAL
Frames: 4
Total Pages: 20
Hits: 11
Faults: 9

Ref String: [8, 5, 2, 5, 8, 9, 3, 9, 2, 3, 8, 4, 2, 7, 4, 4, 6, 2, 5, 7]

OPTIMAL Page Replacement Simulation

	8	5	2	5	8	9	3	9	2	3	8	4	2	7	4	4	6	2	5	7
Frame 1	8	8	8	8	8	8	8	8	8	8	8	4	4	4	4	4	6	6	5	5
Frame 2		5	5	5	5	5	3	3	3	3	3	3	3	7	7	7	7	7	7	7
Frame 3			2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
Frame 4						9	9	9	9	9	9	9	9	9	9	9	9	9	9	9
Status	FAULT	FAULT	FAULT	HIT	HIT	FAULT	FAULT	HIT	HIT	HIT	HIT	FAULT	HIT	FAULT	HIT	HIT	FAULT	HIT	FAULT	HIT
Change	->8	->5	->2	-	-	->9	5->3	-	-	-	-	8->4	-	3->7	-	-	4->6	-	6->5	-

7. Simulate frame allocation Methods. a. minimum number of frames b. equal allocation c. proportional allocation

```
import random
import pandas as pd
import matplotlib.pyplot as plt

n = int(input("Enter number of processes: "))
total_frames = int(input("Enter total number of frames: "))
min_frames = int(input("Enter minimum frames per process: "))

print("\nChoose Allocation Method:")
print("1. Minimum Allocation")
print("2. Equal Allocation")
print("3. Proportional Allocation")

choice = int(input("Enter choice (1/2/3): "))

processes = [f"P{i+1}" for i in range(n)]
sizes = [random.randint(5, 25) for _ in range(n)]

if choice == 1:
    allocated = [min_frames] * n
    title = "Minimum Frame Allocation"

elif choice == 2:
    allocated = [total_frames // n] * n
    title = "Equal Frame Allocation"

elif choice == 3:
    total_size = sum(sizes)
    allocated = [(s * total_frames) // total_size for s in sizes]
    title = "Proportional Frame Allocation"

else:
    print("Invalid choice")
    exit()

used_frames = sum(allocated)
free_frames = total_frames - used_frames
```

```

df = pd.DataFrame({
    "Process": processes,
    "Process_Size(Pages)": sizes,
    "Frames_Allocated": allocated
})

print("\nFrame Allocation Table:\n")
print(df.to_string(index=False))
print(f"\nUnallocated Frames: {free_frames}")

fig, ax = plt.subplots(figsize=(12, 3))

start = 0
colors = plt.cm.tab20.colors

for i in range(n):
    ax.barh(0, allocated[i], left=start, color=colors[i], label=processes[i])
    ax.text(start + allocated[i]/2, 0, processes[i],
            ha="center", va="center", color="white", fontsize=10)
    start += allocated[i]

# Free frames
if free_frames > 0:
    ax.barh(0, free_frames, left=start, color="gray", label="Free")
    ax.text(start + free_frames/2, 0, "Free", ha="center", va="center")

for f in range(total_frames + 1):
    ax.axvline(f, color="black", linewidth=0.3)

ax.set_xlim(0, total_frames)
ax.set_yticks([])
ax.set_xlabel("Frame Number")
ax.set_title(title)

ax.set_xticks(range(0, total_frames + 1))

plt.tight_layout()
plt.show()

```

VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ Date: _____

Output:

Enter number of processes: 5
Enter total number of frames: 20
Enter minimum frames per process: 2

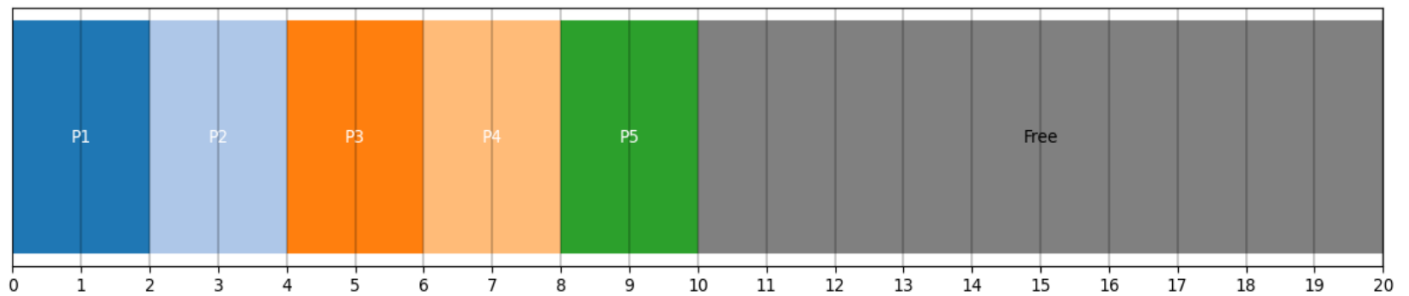
Choose Allocation Method:
1. Minimum Allocation
2. Equal Allocation
3. Proportional Allocation
Enter choice (1/2/3): 1

Frame Allocation Table:

Process	Process_Size(Pages)	Frames_Allocated
P1	13	2
P2	9	2
P3	24	2
P4	21	2
P5	22	2

Unallocated Frames: 10

Minimum Frame Allocation



Enter number of processes: 5
Enter total number of frames: 20
Enter minimum frames per process: 2

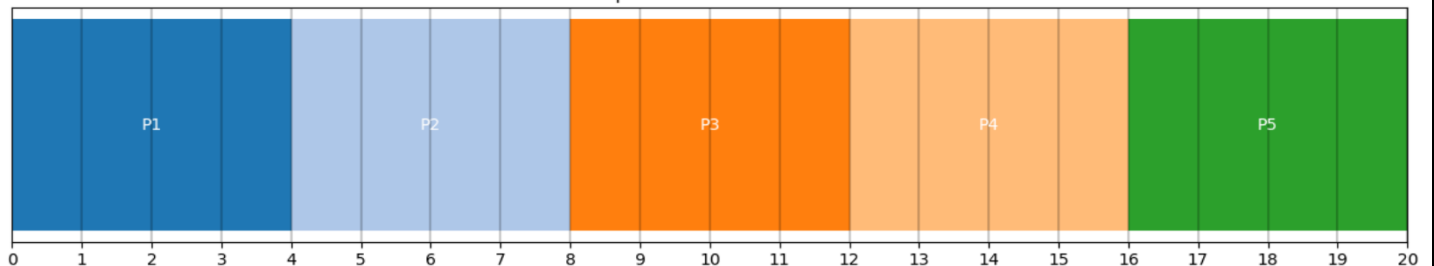
Choose Allocation Method:
1. Minimum Allocation
2. Equal Allocation
3. Proportional Allocation
Enter choice (1/2/3): 2

Frame Allocation Table:

Process	Process_Size(Pages)	Frames_Allocated
P1	18	4
P2	23	4
P3	24	4
P4	17	4
P5	18	4

Unallocated Frames: 0

Equal Frame Allocation



Enter number of processes: 5
Enter total number of frames: 20
Enter minimum frames per process: 2

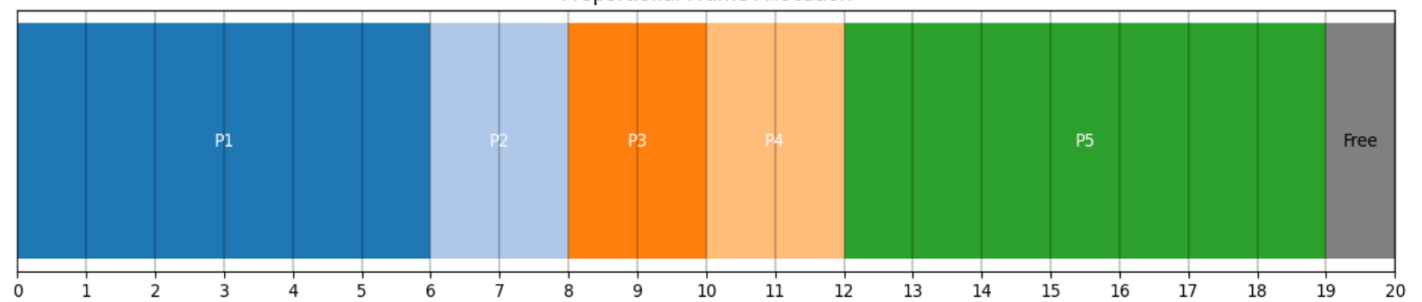
Choose Allocation Method:
1. Minimum Allocation
2. Equal Allocation
3. Proportional Allocation
Enter choice (1/2/3): 3

Frame Allocation Table:

Process	Process_Size(Pages)	Frames_Allocated
P1	19	6
P2	6	2
P3	7	2
P4	6	2
P5	21	7

Unallocated Frames: 1

Proportional Frame Allocation



3. Simulate Bankers Algorithm for Deadlock Avoidance.

```
p = int(input("No of Processes: "))
r = int(input("No of Resources: "))
a = []
m = []
print("Enter Allocation Matrix:")
for i in range(p):
    a.append(list(map(int, input().split()))))

print("Enter Max Matrix:")
for i in range(p):
    m.append(list(map(int, input().split()))))

print("Enter Available Resources:")
av = list(map(int, input().split()))

n = []
for i in range(p):
    row = []
    for j in range(r):
        row.append(m[i][j] - a[i][j])
    n.append(row)

print("\nProcess\tAllocation\tMax\tNeed")
for i in range(p):
    print(f"P {i}\t{a[i]}\t{m[i]}\t{n[i]}")

w = av.copy()
f = [0]*p
seq = []

for _ in range(p):
    ok = False
    for i in range(p):
        if not f[i] and all(n[i][j] <= w[j] for j in range(r)):
            for j in range(r):
                w[j] += a[i][j]
            seq.append(i)
            f[i] = 1
            ok = True
```

```

    if not ok:
        break
if len(seq) == p:
    print("\nSystem is in SAFE state")
    print("Sequence:", " -> ".join(f"P{i}" for i in seq))
else:
    print("\nSystem is NOT in safe state")

```

Output:

```

No of Processes: 5
No of Resources: 3
Enter Allocation Matrix:
0 2 1
1 0 0
1 3 2
0 1 1
2 0 0
Enter Max Matrix:
3 3 2
2 2 2
4 3 3
1 2 2
3 1 1
Enter Available Resources:
2 1 1

```

Process	Allocation	Max	Need
P0	[0, 2, 1]	[3, 3, 2]	[3, 1, 1]
P1	[1, 0, 0]	[2, 2, 2]	[1, 2, 2]
P2	[1, 3, 2]	[4, 3, 3]	[3, 0, 1]
P3	[0, 1, 1]	[1, 2, 2]	[1, 1, 1]
P4	[2, 0, 0]	[3, 1, 1]	[1, 1, 1]

```

System is in SAFE state
Sequence: P3 -> P4 -> P0 -> P1 -> P2

```

```

No of Processes: 5
No of Resources: 3
Enter Allocation Matrix:
1 1 0
1 2 1
1 0 1
0 1 0
2 1 1
Enter Max Matrix:
3 2 2
4 3 3
3 2 2
2 2 2
5 3 3
Enter Available Resources:
2 1 2

```

Process	Allocation	Max	Need
P0	[1, 1, 0]	[3, 2, 2]	[2, 1, 2]
P1	[1, 2, 1]	[4, 3, 3]	[3, 1, 2]
P2	[1, 0, 1]	[3, 2, 2]	[2, 2, 1]
P3	[0, 1, 0]	[2, 2, 2]	[2, 1, 2]
P4	[2, 1, 1]	[5, 3, 3]	[3, 2, 2]

```

System is in SAFE state
Sequence: P0 -> P1 -> P2 -> P3 -> P4

```

8. Simulate the following Disk scheduling algorithms.**a. FCFS****b. SSTF****c. SCAN**

```
import random
import matplotlib.pyplot as plt

def plot_graph(order, title):
    plt.figure(figsize=(12,6))
    plt.plot(order, range(len(order)), marker='o')
    plt.gca().invert_yaxis()
    plt.xlabel("Cylinder Number")
    plt.ylabel("Step Number")
    plt.title(title)
    plt.show()

def fcfs(queue, head):
    order = [head] + queue
    total = 0
    for i in range(len(order)-1):
        total += abs(order[i+1] - order[i])

    print("\nFCFS")
    print("Service Order:", order)
    print("Total Head Movement:", total)
    plot_graph(order, "FCFS Disk Scheduling")

def sstf(queue, head):
    requests = queue.copy()
    current = head
    order = [head]
    total = 0
    for _ in range(len(requests)):
        nearest = requests[0]
        min_dist = abs(nearest - current)
        for r in requests:
            if abs(r - current) < min_dist:
                nearest = r
                min_dist = abs(r - current)
        total += min_dist
        current = nearest
```

```

        order.append(nearest)
        requests.remove(nearest)

    print("\nSSTF")
    print("Service Order:", order)
    print("Total Head Movement:", total)
    plot_graph(order, "SSTF Disk Scheduling")

def scan(queue, head, start, end, direction):
    left = [x for x in queue if x < head]
    right = [x for x in queue if x >= head]

    left.sort()
    right.sort()
    order = [head]
    if direction == "left":
        left.reverse()
        order += left
        order.append(start)
        order += right
    else:
        order += right
        order.append(end)
        left.reverse()
        order += left
    total = 0
    for i in range(len(order)-1):
        total += abs(order[i+1] - order[i])

    print("\nSCAN")
    print("Service Order:", order)
    print("Total Head Movement:", total)
    plot_graph(order, "SCAN Disk Scheduling")

start = int(input("Enter start cylinder: "))
end = int(input("Enter end cylinder: "))
qsize = int(input("Enter queue size: "))

queue = [random.randint(start, end) for _ in range(qsize)]
head = random.randint(start, end)

print("\nGenerated Queue:", queue)
print("Head Position:", head)

```



```
print("\n----- Disk Scheduling Menu -----")
print("1. FCFS")
print("2. SSTF")
print("3. SCAN")
choice = int(input("Enter choice: "))
```

```
if choice == 1:
    fcfs(queue, head)
elif choice == 2:
    sstf(queue, head)
elif choice == 3:
    print("\nDirection:")
    print("1. Left")
    print("2. Right")
    d = input("Enter direction: ")
```

```
if d == "1":
    direction = "left"
elif d == "2":
    direction = "right"
else:
    print("Invalid direction")
    exit()
```

```
scan(queue, head, start, end, direction)
else:
    print("Invalid choice")
```

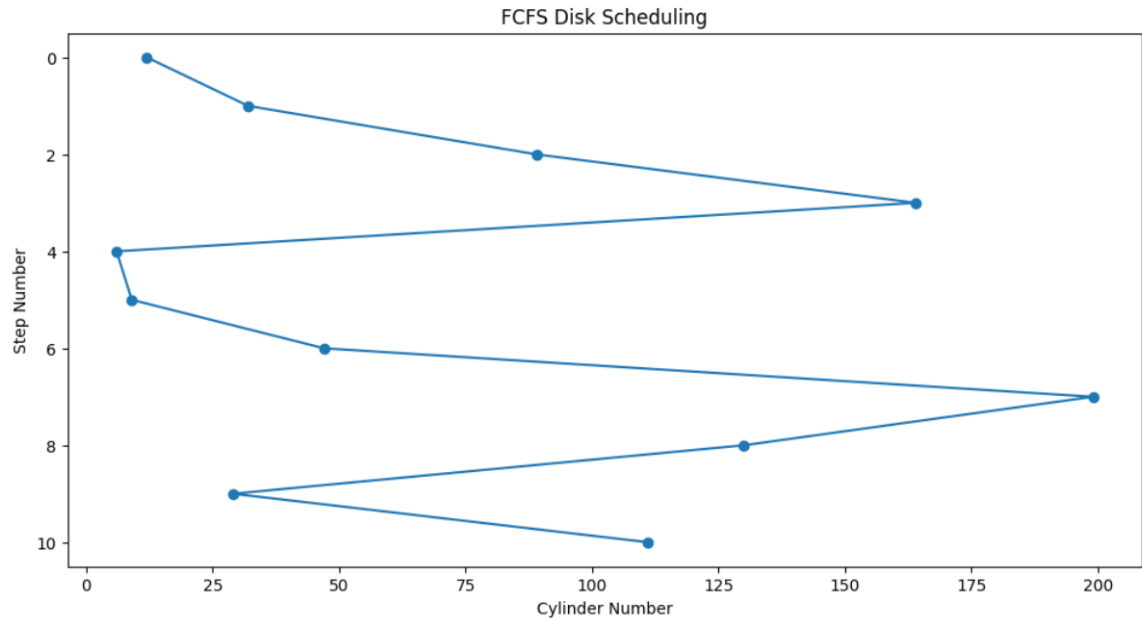
Output:

Enter start cylinder: 0
Enter end cylinder: 199
Enter queue size: 10

Generated Queue: [32, 89, 164, 6, 9, 47, 199, 130, 29, 111]
Head Position: 12

----- Disk Scheduling Menu -----
1. FCFS
2. SSTF
3. SCAN
Enter choice: 1

FCFS
Service Order: [12, 32, 89, 164, 6, 9, 47, 199, 130, 29, 111]
Total Head Movement: 755

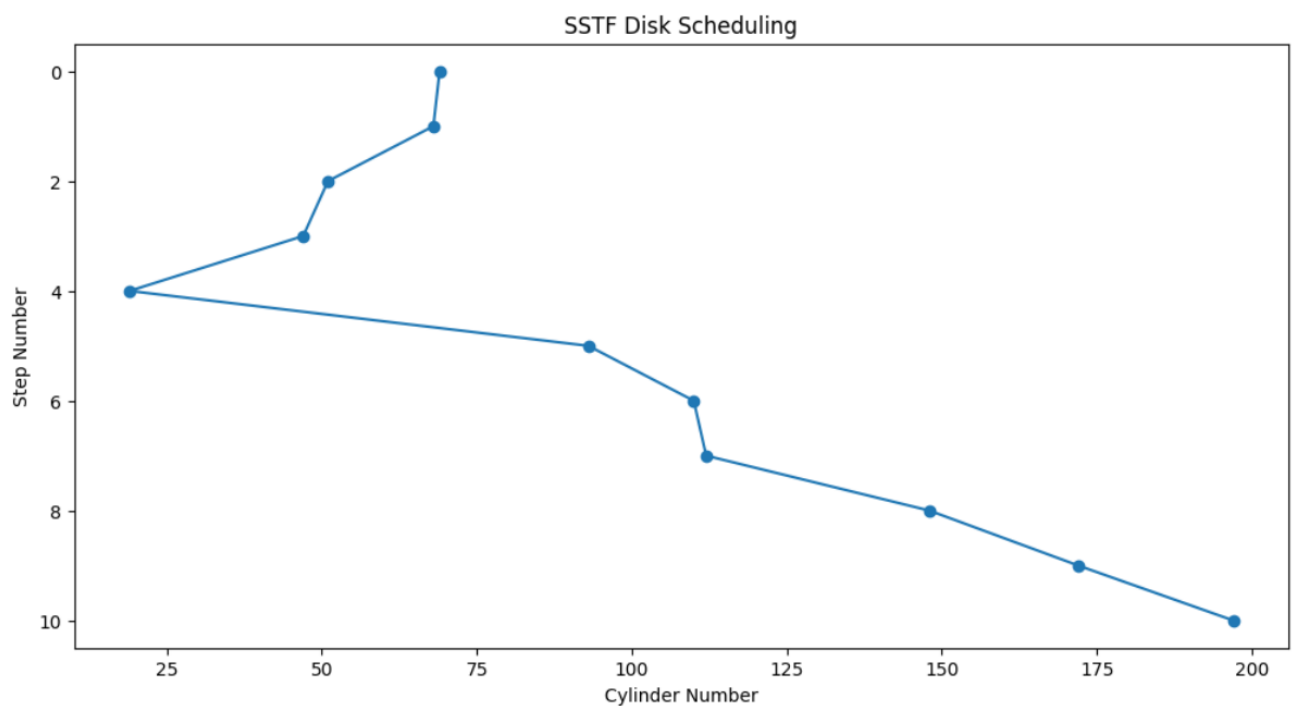


Enter start cylinder: 0
Enter end cylinder: 199
Enter queue size: 10

Generated Queue: [112, 110, 19, 68, 172, 93, 51, 197, 148, 47]
Head Position: 69

----- Disk Scheduling Menu -----
1. FCFS
2. SSTF
3. SCAN
Enter choice: 2

SSTF
Service Order: [69, 68, 51, 47, 19, 93, 110, 112, 148, 172, 197]
Total Head Movement: 228



Enter start cylinder: 0
Enter end cylinder: 199
Enter queue size: 10

Generated Queue: [144, 36, 61, 92, 125, 84, 183, 91, 178, 8]
Head Position: 116

----- Disk Scheduling Menu -----

1. FCFS
2. SSTF
3. SCAN

Enter choice: 3

Direction:

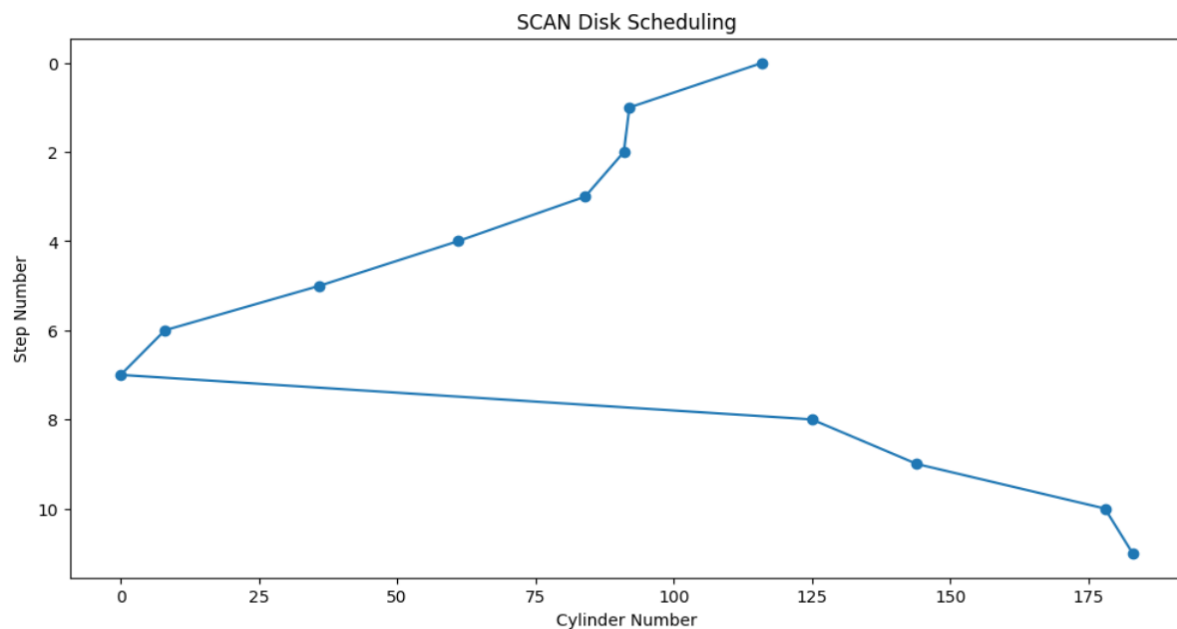
1. Left
2. Right

Enter direction: 1

SCAN

Service Order: [116, 92, 91, 84, 61, 36, 8, 0, 125, 144, 178, 183]

Total Head Movement: 299



Enter start cylinder: 0
Enter end cylinder: 199
Enter queue size: 10

Generated Queue: [14, 9, 158, 37, 109, 117, 62, 195, 42, 114]
Head Position: 141

----- Disk Scheduling Menu -----

1. FCFS
2. SSTF
3. SCAN

Enter choice: 3

Direction:

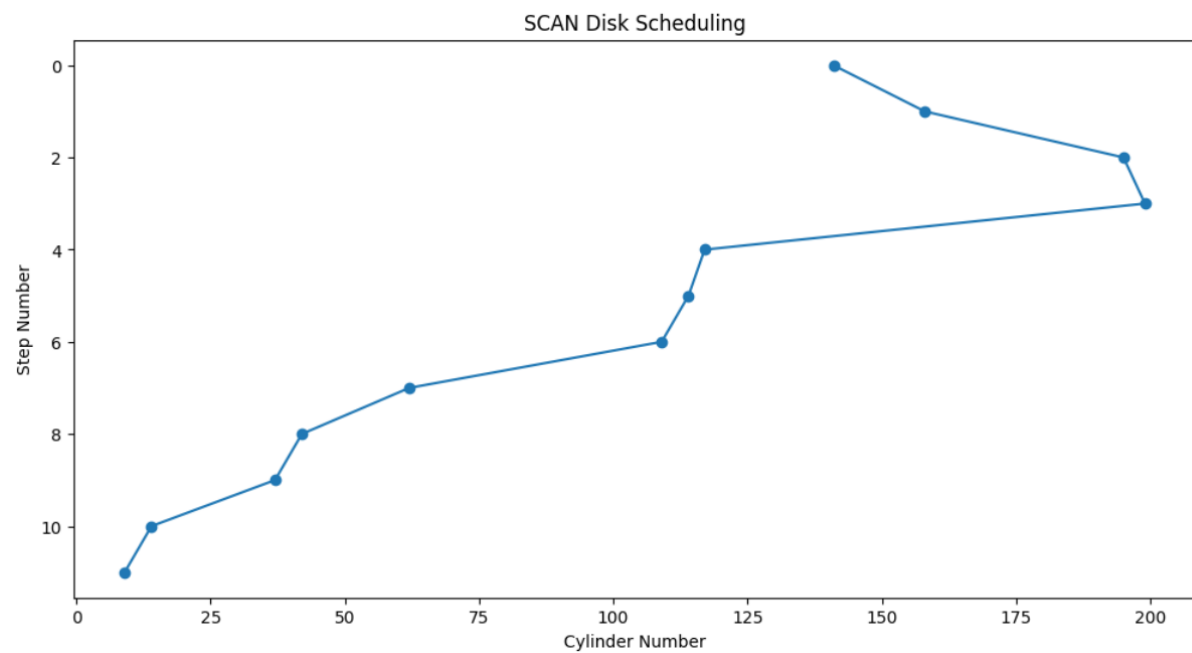
1. Left
2. Right

Enter direction: 2

SCAN

Service Order: [141, 158, 195, 199, 117, 114, 109, 62, 42, 37, 14, 9]

Total Head Movement: 248



9. Simulate the following Disk scheduling algorithms.

a. C-SCAN

b. LOOK

c. C-LOOK

```
import random
import matplotlib.pyplot as plt

def plot_graph(order, title):
    plt.figure(figsize=(12, 6))
    y = [i/2 for i in range(len(order))]
    plt.plot(order, y, marker='o')
    plt.gca().invert_yaxis()
    plt.xlabel("Cylinder Number")
    plt.ylabel("Step Number")
    plt.title(title)
    plt.show()

def cscan(queue, head, start, end, direction):
    left = sorted([x for x in queue if x < head])
    right = sorted([x for x in queue if x >= head])

    order = [head]

    if direction == "right":
        order += right
        order.append(end)
        order.append(start)
        order += left

    else: # left
        order += left[::-1]
        order.append(start)
        order.append(end)
        order += right[::-1]

    total = 0
    for i in range(len(order)-1):
        total += abs(order[i+1] - order[i])

    print("\nC-SCAN ")
    print("Service Order:", order)
```

```

print("Total Head Movement:", total)

plot_graph(order, "C-SCAN Disk Scheduling")

def look(queue, head, direction):

    left = sorted([x for x in queue if x < head])
    right = sorted([x for x in queue if x >= head])

    order = [head]

    if direction == "right":
        order += right
        order += left[::-1]
    else:
        order += left[::-1]
        order += right

    total = 0
    for i in range(len(order)-1):
        total += abs(order[i+1] - order[i])

    print("\nLOOK")
    print("Service Order:", order)
    print("Total Head Movement:", total)

    plot_graph(order, "LOOK Disk Scheduling")

def clook(queue, head, direction):

    left = sorted([x for x in queue if x < head])
    right = sorted([x for x in queue if x >= head])

    order = [head]

    if direction == "right":
        order += right
        order += left
    else:
        order += left[::-1]
        order += right[::-1]
    total = 0

```

```
for i in range(len(order)-1):
    total += abs(order[i+1] - order[i])

print("\nC-LOOK")
print("Service Order:", order)
print("Total Head Movement:", total)
plot_graph(order, "C-LOOK Disk Scheduling")

start = int(input("Enter start cylinder: "))
end = int(input("Enter end cylinder: "))
qsize = int(input("Enter queue size: "))

queue = [random.randint(start, end) for _ in range(qsize)]
head = random.randint(start, end)
print("\nQueue:", queue)
print("Head:", head)
print("\n----- MENU -----")
print("1. C-SCAN")
print("2. LOOK")
print("3. C-LOOK")
choice = int(input("Enter choice: "))
print("\nDirection:")
print("1. Left")
print("2. Right")
d = int(input("Enter direction: "))
if d == 1:
    direction = "left"
elif d == 2:
    direction = "right"
else:
    print("Invalid direction")
    exit()
if choice == 1:
    cscan(queue, head, start, end, direction)
elif choice == 2:
    look(queue, head, direction)
elif choice == 3:
    clook(queue, head, direction)
else:
    print("Invalid choice")
```

Enter start cylinder: 0
Enter end cylinder: 199
Enter queue size: 10

Queue: [100, 73, 43, 181, 15, 91, 3, 149, 138, 79]
Head: 116

----- MENU -----

1. C-SCAN
2. LOOK
3. C-LOOK

Enter choice: 1

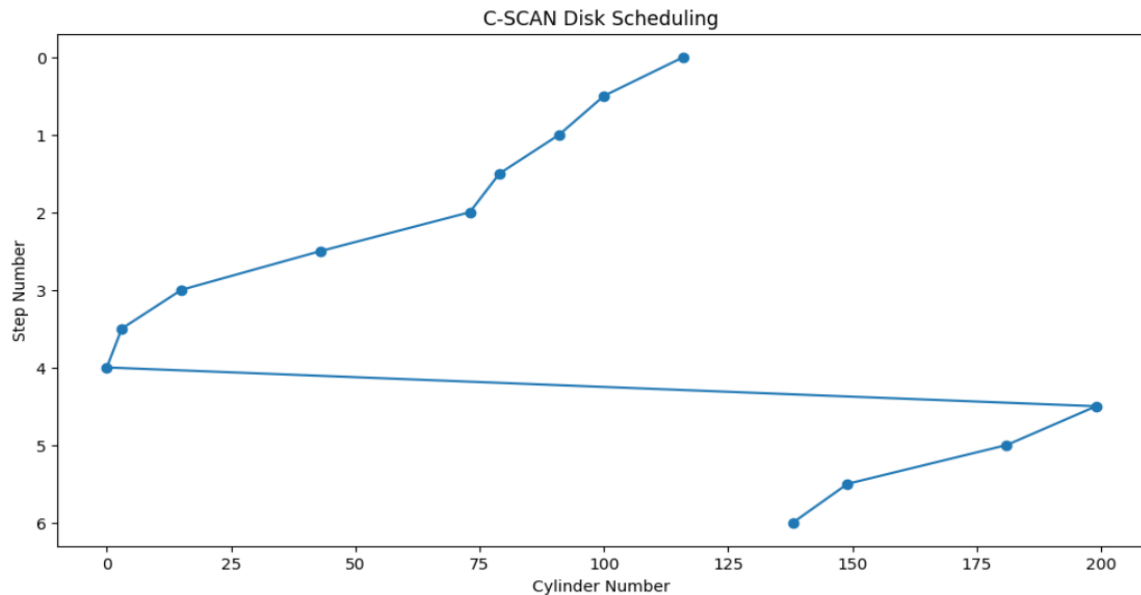
Direction:

1. Left
2. Right

Enter direction: 1

C-SCAN

Service Order: [116, 100, 91, 79, 73, 43, 15, 3, 0, 199, 181, 149, 138]
Total Head Movement: 376



Enter start cylinder: 0
Enter end cylinder: 199
Enter queue size: 10

Queue: [53, 42, 107, 110, 74, 27, 25, 32, 64, 97]
Head: 116

----- MENU -----

1. C-SCAN
2. LOOK
3. C-LOOK

Enter choice: 1

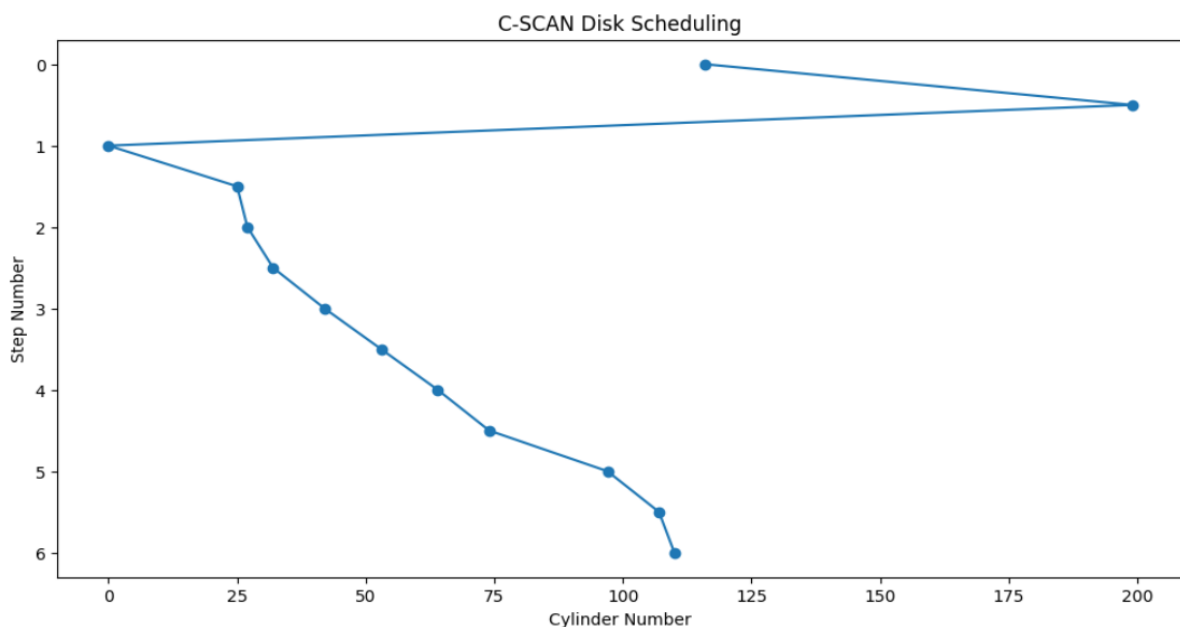
Direction:

1. Left
2. Right

Enter direction: 2

C-SCAN

Service Order: [116, 199, 0, 25, 27, 32, 42, 53, 64, 74, 97, 107, 110]
Total Head Movement: 392



VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ Date: _____

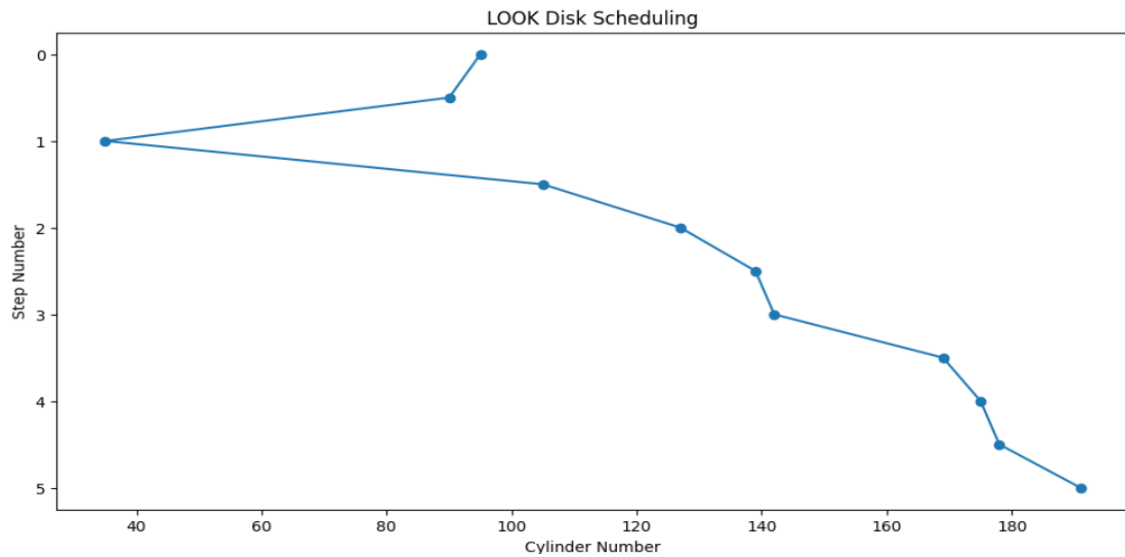
```
Enter start cylinder: 0
Enter end cylinder: 199
Enter queue size: 10
```

```
Queue: [127, 191, 35, 142, 169, 178, 175, 90, 139, 105]
Head: 95
```

```
----- MENU -----
1. C-SCAN
2. LOOK
3. C-LOOK
Enter choice: 2
```

```
Direction:
1. Left
2. Right
Enter direction: 1
```

```
LOOK
Service Order: [95, 90, 35, 105, 127, 139, 142, 169, 175, 178, 191]
Total Head Movement: 216
```



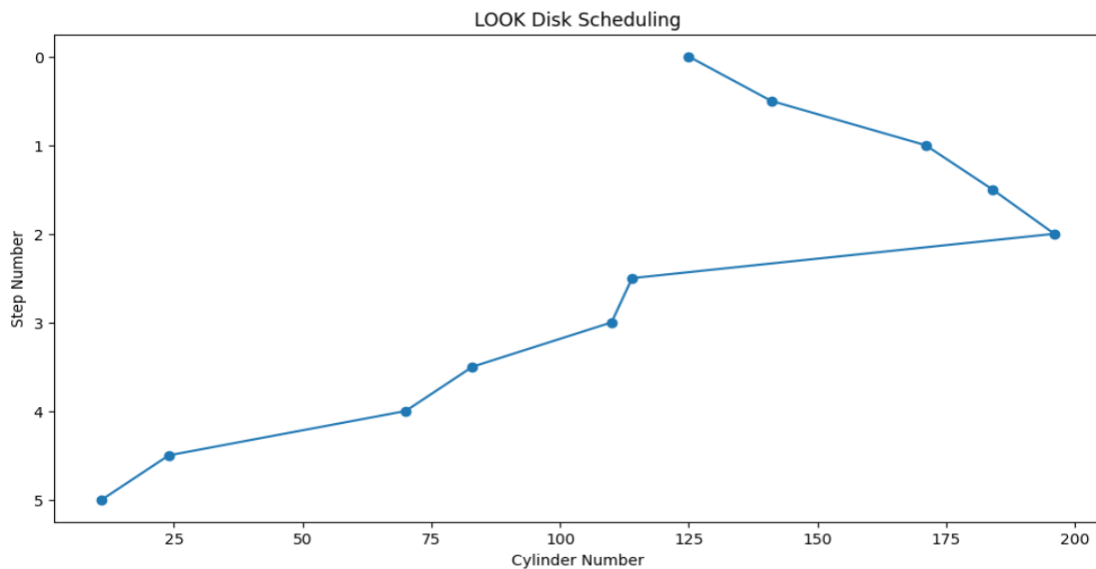
```
Enter start cylinder: 0
Enter end cylinder: 199
Enter queue size: 10
```

```
Queue: [171, 24, 114, 110, 70, 196, 83, 141, 11, 184]
Head: 125
```

```
----- MENU -----
1. C-SCAN
2. LOOK
3. C-LOOK
Enter choice: 2
```

```
Direction:
1. Left
2. Right
Enter direction: 2
```

```
LOOK
Service Order: [125, 141, 171, 184, 196, 114, 110, 83, 70, 24, 11]
Total Head Movement: 256
```

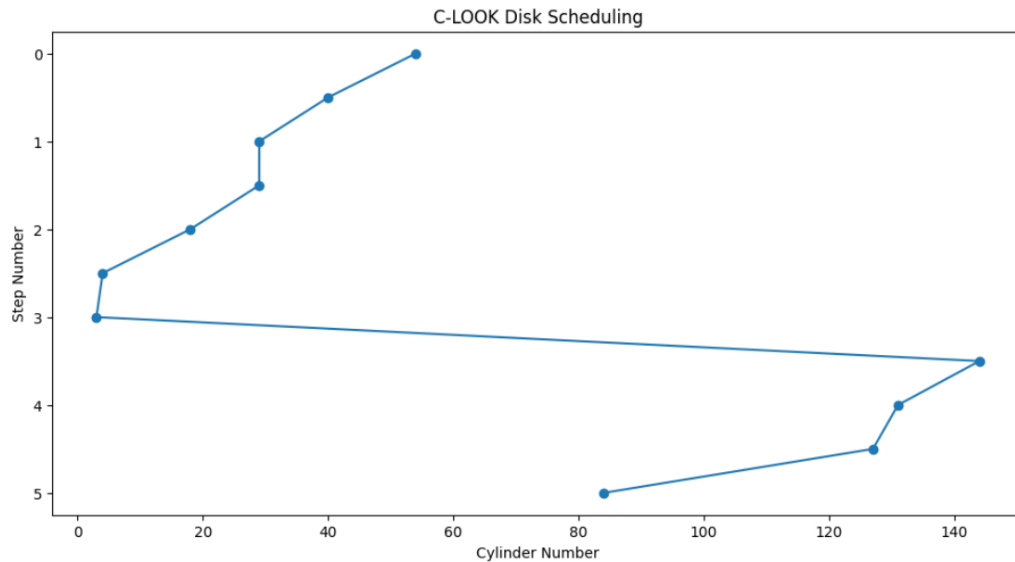


Enter start cylinder: 0
 Enter end cylinder: 199
 Enter queue size: 10
 Queue: [4, 40, 29, 3, 144, 131, 127, 84, 29, 18]
 Head: 54

----- MENU -----
 1. C-SCAN
 2. LOOK
 3. C-LOOK
 Enter choice: 3

Direction:
 1. Left
 2. Right
 Enter direction: 1

C-LOOK
 Service Order: [54, 40, 29, 29, 18, 4, 3, 144, 131, 127, 84]
 Total Head Movement: 252

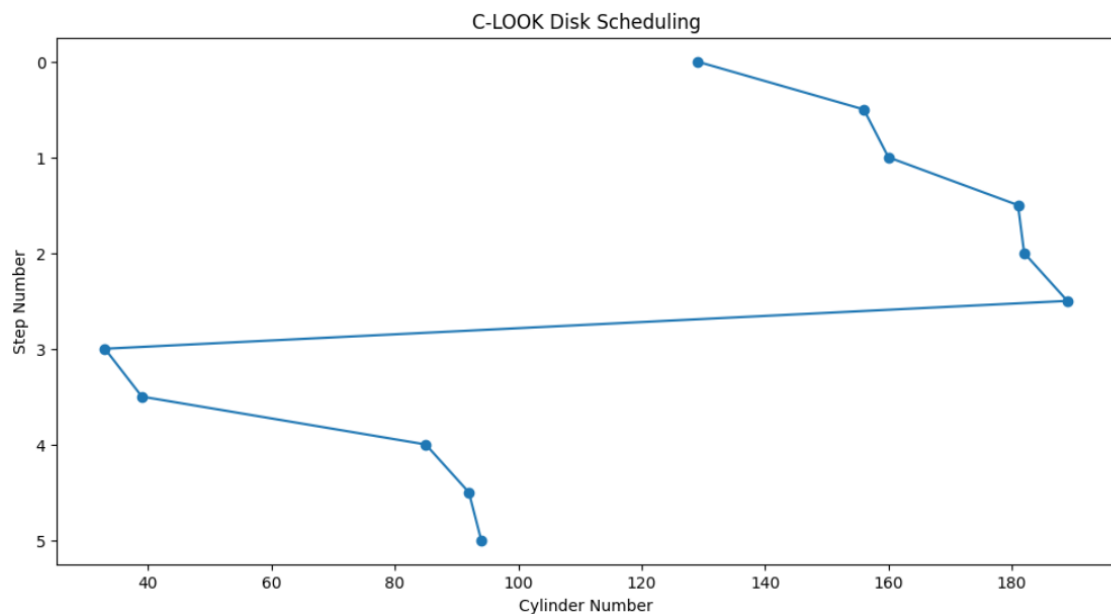


Enter start cylinder: 0
 Enter end cylinder: 199
 Enter queue size: 10
 Queue: [189, 160, 33, 92, 182, 85, 156, 181, 39, 94]
 Head: 129

----- MENU -----
 1. C-SCAN
 2. LOOK
 3. C-LOOK
 Enter choice: 3

Direction:
 1. Left
 2. Right
 Enter direction: 2

C-LOOK
 Service Order: [129, 156, 160, 181, 182, 189, 33, 39, 85, 92, 94]
 Total Head Movement: 277



10. Simulate all file allocation strategies. a) Sequential b) Indexed c) Linked.

```
import random

n = int(input("Enter number of locations: "))
disk = [False] * n

for i in range(n):
    if random.random() < 0.4:
        disk[i] = True

print("\nInitial Disk State:")
print(["1" if x else "0" for x in disk])

print("\nChoose Allocation Method:")
print("1) Sequential Allocation")
print("2) Linked Allocation")
print("3) Indexed Allocation")

choice = int(input("Enter choice: "))
num_files = int(input("Enter number of files: "))

for f in range(1, num_files + 1):
    print(f"\n--- Allocating File F {f} ---")
    size = random.randint(1, 5)
    print("File size:", size)

    if choice == 1:
        allocated = False
        for i in range(n - size + 1):
            if not any(disk[i:i+size]):
                for j in range(i, i+size):
                    disk[j] = True

                print("Allocated blocks:", list(range(i, i+size)))
                print(f"Stored at positions: {i} to {i+size-1}")
                allocated = True
                break

        if not allocated:
            print("Allocation failed (no contiguous space)")

    elif choice == 2:
        free_blocks = [i for i in range(n) if not disk[i]]
```

```

if len(free_blocks) < size:
    print("Allocation failed (not enough space)")
else:
    blocks = random.sample(free_blocks, size)

    for b in blocks:
        disk[b] = True

    print("Allocated blocks (linked):")
    print(" -> ".join(map(str, blocks)))
    print("Stored at positions:", blocks)

elif choice == 3:
    free_blocks = [i for i in range(n) if not disk[i]]

    if len(free_blocks) < size + 1:
        print("Allocation failed (need index + data blocks)")
    else:
        index_block = free_blocks[0]
        data_blocks = random.sample(free_blocks[1:], size)

        disk[index_block] = True
        for b in data_blocks:
            disk[b] = True

        print("Index Block:", index_block)
        print("Data Blocks:", data_blocks)
        print(f"{index_block} -> {data_blocks}")

else:
    print("Invalid choice")

print("Disk state:", ["1" if x else "0" for x in disk])

```

Output:

VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ Date: _____

Enter number of locations: 20

Initial Disk State:

['0', '1', '0', '0', '1', '0', '0', '0', '1', '0', '0', '1', '0', '0', '1', '0', '0', '0', '0', '0']

Choose Allocation Method:

1) Sequential Allocation

2) Linked Allocation

3) Indexed Allocation

Enter choice: 1

Enter number of files: 3

--- Allocating File F1 ---

File size: 3

Allocated blocks: [5, 6, 7]

Stored at positions: 5 to 7

Disk state: ['0', '1', '0', '0', '1', '1', '1', '1', '1', '0', '0', '1', '0', '0', '1', '0', '0', '0', '0', '0']

--- Allocating File F2 ---

File size: 5

Allocated blocks: [15, 16, 17, 18, 19]

Stored at positions: 15 to 19

Disk state: ['0', '1', '0', '0', '1', '1', '1', '1', '1', '0', '0', '1', '0', '0', '1', '1', '1', '1', '1', '1']

--- Allocating File F3 ---

File size: 3

Allocation failed (no contiguous space)

Disk state: ['0', '1', '0', '0', '1', '1', '1', '1', '1', '0', '0', '1', '0', '0', '1', '1', '1', '1', '1', '1']

Enter number of locations: 20

Initial Disk State:

['1', '0', '1', '1', '0', '0', '0', '0', '0', '0', '0', '0', '1', '0', '0', '1', '0', '0', '1', '0']

Choose Allocation Method:

1) Sequential Allocation

2) Linked Allocation

3) Indexed Allocation

Enter choice: 2

Enter number of files: 3

--- Allocating File F1 ---

File size: 3

Allocated blocks (linked):

6 -> 8 -> 5

Stored at positions: [6, 8, 5]

Disk state: ['1', '0', '1', '1', '0', '1', '1', '0', '1', '0', '0', '0', '1', '0', '0', '1', '0', '0', '1', '0']

--- Allocating File F2 ---

File size: 4

Allocated blocks (linked):

7 -> 14 -> 19 -> 4

Stored at positions: [7, 14, 19, 4]

Disk state: ['1', '0', '1', '1', '1', '1', '1', '1', '1', '0', '0', '0', '1', '0', '1', '1', '0', '0', '1', '1']

--- Allocating File F3 ---

File size: 5

Allocated blocks (linked):

17 -> 11 -> 9 -> 16 -> 10

Stored at positions: [17, 11, 9, 16, 10]

Disk state: ['1', '0', '1', '1', '1', '1', '1', '1', '1', '1', '1', '1', '0', '1', '1', '1', '1', '1', '1', '1']

Enter number of locations: 20

Initial Disk State:

['0', '0', '0', '1', '0', '1', '0', '1', '0', '0', '0', '0', '0', '0', '1', '0', '0', '0', '0']

Choose Allocation Method:

1) Sequential Allocation

2) Linked Allocation

3) Indexed Allocation

Enter choice: 3

Enter number of files: 3

--- Allocating File F1 ---

File size: 3

Index Block: 0

Data Blocks: [2, 1, 14]

0 -> [2, 1, 14]

Disk state: ['1', '1', '1', '1', '0', '1', '0', '1', '0', '0', '0', '0', '0', '0', '1', '1', '0', '0', '0']

--- Allocating File F2 ---

File size: 1

Index Block: 4

Data Blocks: [19]

4 -> [19]

Disk state: ['1', '1', '1', '1', '1', '1', '0', '1', '0', '0', '0', '0', '0', '0', '1', '1', '0', '0', '1']

--- Allocating File F3 ---

File size: 1

Index Block: 6

Data Blocks: [8]

6 -> [8]

Disk state: ['1', '1', '1', '1', '1', '1', '1', '1', '1', '0', '0', '0', '0', '0', '1', '1', '0', '0', '1']

5. Implementation of Paging Technique of Memory Management

```
import random
import math

page_size = int(input("Enter page (frame) size: "))
process_size = int(input("Enter process size: "))
memory_size = int(input("Enter main memory size: "))

if page_size <= 0 or process_size <= 0 or memory_size <= 0:
    print("Error: All inputs must be positive!")
    exit()

if (page_size & (page_size - 1)) != 0:
    print("Error: Page size must be a power of 2!")
    exit()

if memory_size % page_size != 0:
    print("Error: Memory size must be a multiple of page size!")
    exit()

if process_size > memory_size:
    print("Error: Process size should be less than or equal to main memory size!")
    exit()

num_pages = (process_size + page_size - 1) // page_size
num_frames = memory_size // page_size

if num_pages > num_frames:
    print("Error: Not enough memory to allocate all pages!")
    exit()

offset_bits = int(math.log2(page_size))
page_bits = int(math.ceil(math.log2(num_pages)))
frame_bits = int(math.ceil(math.log2(num_frames)))
physical_bits = frame_bits + offset_bits

print("\nNumber of pages:", num_pages)
print("Number of frames:", num_frames)

free_frames = list(range(num_frames))
page_table = {}

for i in range(num_pages):
    frame = random.choice(free_frames)
    page_table[i] = frame
    free_frames.remove(frame)
```

```

print("\nPage Table (Random Allocation):")
for p in page_table:
    print(f'Page {p} -> Frame {page_table[p]}')

print("\nByte -> (Page, Offset) -> Frame -> Physical Address:\n")

for byte in range(process_size):
    page_no = byte // page_size
    offset = byte % page_size
    frame_no = page_table[page_no]

    physical_address = frame_no * page_size + offset

    byte_bin = format(byte, f'0{page_bits + offset_bits}b')
    page_bin = format(page_no, f'0{page_bits}b')
    offset_bin = format(offset, f'0{offset_bits}b')
    frame_bin = format(frame_no, f'0{frame_bits}b')
    physical_bin = format(physical_address, f'0{physical_bits}b')

    print(f'Byte {byte} ({byte_bin}) -> "
        f'(Page {page_no} ({page_bin}), Offset {offset} ({offset_bin})) -> "
        f'Frame {frame_no} ({frame_bin}) -> "
        f'Physical Address {physical_address} ({physical_bin})"')

```

Output:

```

Enter page (frame) size: 2
Enter process size: 8
Enter main memory size: 32

```

```

Number of pages: 4
Number of frames: 16

```

```

Page Table (Random Allocation):
Page 0 -> Frame 13
Page 1 -> Frame 9
Page 2 -> Frame 6
Page 3 -> Frame 12

```

```

Byte -> (Page, Offset) -> Frame -> Physical Address:

```

```

Byte 0 (000) -> (Page 0 (00), Offset 0 (0)) -> Frame 13 (1101) -> Physical Address 26 (11010)
Byte 1 (001) -> (Page 0 (00), Offset 1 (1)) -> Frame 13 (1101) -> Physical Address 27 (11011)
Byte 2 (010) -> (Page 1 (01), Offset 0 (0)) -> Frame 9 (1001) -> Physical Address 18 (10010)
Byte 3 (011) -> (Page 1 (01), Offset 1 (1)) -> Frame 9 (1001) -> Physical Address 19 (10011)
Byte 4 (100) -> (Page 2 (10), Offset 0 (0)) -> Frame 6 (0110) -> Physical Address 12 (01100)
Byte 5 (101) -> (Page 2 (10), Offset 1 (1)) -> Frame 6 (0110) -> Physical Address 13 (01101)
Byte 6 (110) -> (Page 3 (11), Offset 0 (0)) -> Frame 12 (1100) -> Physical Address 24 (11000)
Byte 7 (111) -> (Page 3 (11), Offset 1 (1)) -> Frame 12 (1100) -> Physical Address 25 (11001)

```


VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ Date: _____

Enter page (frame) size: 2

Enter process size: 4

Enter main memory size: 16

Number of pages: 2

Number of frames: 8

Page Table (Random Allocation):

Page 0 -> Frame 3

Page 1 -> Frame 5

Byte -> (Page, Offset) -> Frame -> Physical Address:

Byte 0 (00) -> (Page 0 (0), Offset 0 (0)) -> Frame 3 (011) -> Physical Address 6 (0110)

Byte 1 (01) -> (Page 0 (0), Offset 1 (1)) -> Frame 3 (011) -> Physical Address 7 (0111)

Byte 2 (10) -> (Page 1 (1), Offset 0 (0)) -> Frame 5 (101) -> Physical Address 10 (1010)

Byte 3 (11) -> (Page 1 (1), Offset 1 (1)) -> Frame 5 (101) -> Physical Address 11 (1011)